1 Introduction

Direct marketing campaigns are widely used by banks to promote financial products, but contacting every customer is costly and often inefficient. The goal of this project is to build a binary classification model that predicts whether a client will subscribe to a term deposit ("yes" or "no") based on customer and campaign-related information. A strong model can help identify high-probability customers more efficiently and reduce unnecessary outreach. To solve this problem, I compared two different modeling strategies: Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP).

This project uses the UCI Bank Marketing dataset, which contains 45,211 customer records. The target variable is imbalanced: 5,289 customers subscribed to a term deposit, while 39,922 did not, so the positive class represents only about 11.7% of the data. The dataset includes a mix of numerical and categorical features, such as age, account balance, education, job, marital status, housing and loan status, contact type, month of contact, campaign frequency, previous contact history, and previous campaign outcome. The dataset is a good fit for this comparison because it contains both numerical and categorical features, some missing values, and relationships that may be nonlinear.

2 Methods

The raw dataset was first cleaned to ensure that all features had consistent data types and that missing values were handled appropriately. Numerical columns such as age, balance, duration, campaign, pdays, and previous were converted to numeric form, while categorical columns such as job, marital status, education, housing, loan status, contact type, month, and previous campaign outcome were treated as categorical variables. Missing categorical entries were preserved and later imputed during preprocessing rather than dropped, since removing those rows would unnecessarily reduce the amount of training data.

I also created a few additional engineered features to capture potentially useful structure in the data. These included a binary indicator for whether a client had been contacted previously, an age-group feature based on age ranges, and a ratio comparing the current campaign count to the number of previous contacts. These features were added to test whether simple domain-informed transformations could improve model performance beyond the original variables alone.

The dataset was split into training, validation, and test sets using a 70/15/15 split with stratification on the target variable so that each subset preserved the original class distribution. This was especially important because the dataset is imbalanced, with the positive class representing only a small fraction of the observations. All preprocessing steps were fit only on the training data to avoid data leakage. The validation set was used for hyperparameter tuning and threshold selection, while the test set was reserved for final evaluation.

For preprocessing, numerical features were imputed using the median, while categorical features were imputed with a constant placeholder value and then one-hot encoded. For the MLP model, numerical features were additionally standardized using StandardScaler. This scaling step was necessary because neural networks are sensitive to feature magnitude, and unscaled inputs can make optimization slower and less stable. In contrast, GBDT does not require feature scaling because tree-based splits depend on feature ordering rather than distance or gradient magnitude.

For the GBDT model, I used XGBClassifier and explored several key hyperparameters, including learning_rate, n_estimators, max_depth, subsample, reg_alpha, and reg_lambda. Model selection was based primarily on validation F1 score because the dataset is imbalanced, and F1 provides a better balance between precision and recall than accuracy alone. I also used early stopping during monitored training runs to reduce overfitting and to compare training and validation loss across boosting rounds. In addition, feature importance plots and learning-rate comparisons were used to better understand model behavior.

Finally, both model families were evaluated using accuracy, precision, recall, F1 score, and area under the precision-recall curve. These metrics were chosen to provide a more complete view of performance under class imbalance. After tuning, the final GBDT and MLP models were compared side by side on the held-out test set.

3 Results

The initial baseline models provided a starting point for comparing the two model families. On the validation set, the baseline GBDT achieved an accuracy of 0.9101, precision of 0.6603, recall of 0.4773, F1 score of 0.5541, and AUPRC of 0.6393. The baseline MLP achieved an accuracy of 0.9086, precision of 0.6399, recall of 0.5013, F1 score of 0.5621, and AUPRC of 0.6145. These baseline results showed that both models performed reasonably well, with GBDT producing slightly stronger precision and AUPRC, while MLP achieved slightly higher recall and F1.

After hyperparameter tuning, both models improved, although the gains differed by metric. For GBDT, the best validation configuration used a relatively higher learning rate with regularization, and the monitored training runs showed clear differences in validation loss across learning-rate settings. For MLP, tuning produced only modest gains over the baseline, and performance depended more strongly on the specific architecture and learning-rate combination. The training loss curve for MLP showed that the network converged steadily without obvious instability. In addition, tuning the decision threshold for both models changed the balance between precision and recall, which was especially important because the dataset is imbalanced.

The visualizations provided additional insight into model behavior. The GBDT training-versus-validation loss plot (Figure 1) showed how performance changed across boosting rounds and helped identify overfitting behavior. The learning-rate comparison plot (Figure 2) demonstrated that different learning rates produced noticeably different validation-loss trajectories, showing the tradeoff between slower learning and more aggressive fitting. The feature-importance plot (Figure 3) indicated that previous campaign outcome, contact information, duration, and several month-related indicators were among the most influential predictors. For MLP, the training loss curve (Figure 4) showed the optimization process over iterations, while the depth-and-width comparison plot (Figure 5) illustrated how network architecture affected validation F1 score.

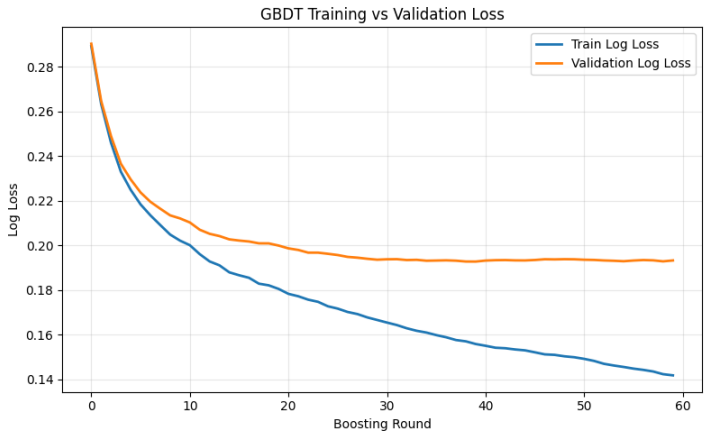

Figure 1: GBDT training and validation log loss across boosting rounds.

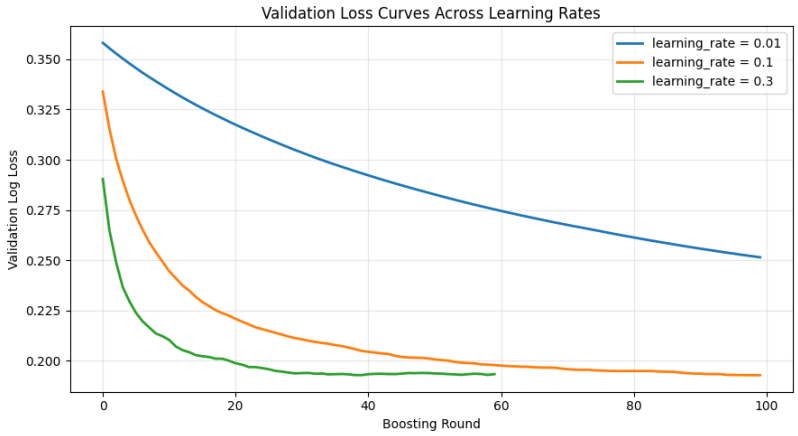

Figure 2: GBDT validation loss across learning-rate settings.

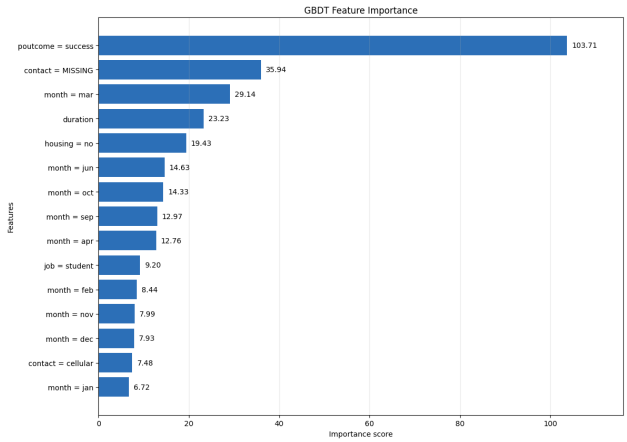

Figure 3: Top GBDT feature importances based on gain.

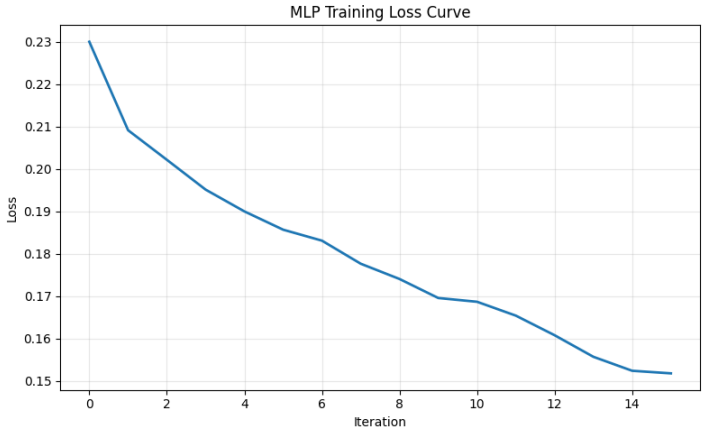

Figure 4: MLP training loss across iterations.

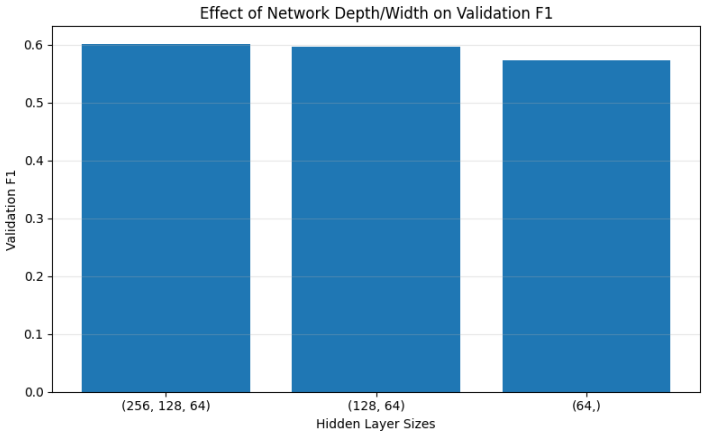

Figure 5: Best validation F1 achieved by each tested MLP architecture.

On the final test set, the tuned GBDT achieved an accuracy of 0.8969, precision of 0.5445, recall of 0.7251, F1 score of 0.6220, and AUPRC of 0.6070. The tuned MLP achieved an accuracy of 0.8888, precision of 0.5166, recall of 0.7667, F1 score of 0.6173, and AUPRC of 0.5823. Overall, GBDT produced the stronger accuracy, precision, F1, and AUPRC, while MLP achieved the stronger recall (Figure 6). These results suggest that after threshold tuning, the MLP became more aggressive at identifying positive cases, while GBDT still provided the better overall balance between precision and recall.

GBDT vs MLP Comparison

model	accuracy	precision	recall	f1	auc_pr	train_time_sec
GBDT	0.8969	0.5445	0.7251	0.622	0.607	0.1584
MLP	0.8888	0.5166	0.7667	0.6173	0.5823	0.8712

Figure 6: Final test-set comparison of tuned GBDT and MLP models.

4 GBDT vs MLP Comparison

The final results show that the two model families performed well, but they emphasized different aspects of the classification task. On the test set, GBDT achieved higher accuracy, precision, F1 score, and AUPRC, while MLP achieved higher recall. This means that the tuned MLP identified more customers who would actually subscribe to a term deposit, but it did so at the cost of more false positives. Given the class imbalance in the dataset, the higher recall of MLP is important, but GBDT's slightly stronger F1 score and higher AUPRC suggest that it provided the better overall balance on this task.

One important difference between the models is how they learn from structured tabular data. GBDT naturally handles nonlinear relationships, threshold effects, and interactions between categorical and numerical variables with relatively little preprocessing. This is likely one reason it performed strongly on this dataset, which contains mixed feature types, missing values, and variables whose relationships with the target may be complex and nonlinear. In contrast, the MLP required more preprocessing, especially feature scaling, and its performance depended more heavily on architecture and optimization choices. Although the threshold-tuned MLP achieved the highest recall, GBDT still performed better on the broader set of evaluation metrics.

The training behavior of the two models also reflected different bias-variance tradeoffs. For GBDT, the validation-loss and hyperparameter comparison plots showed that increasing model complexity could improve performance, but it also increased the risk of overfitting. Parameters such as learning rate, tree depth, and number of estimators had a visible effect on the gap between training and validation performance. For MLP, the training-loss curve was smoother, and performance changes were more closely tied to architecture size and learning rate. Some deeper MLP configurations achieved strong validation F1 scores, but the model was more sensitive to tuning choices and less consistent across evaluation settings than GBDT.

Interpretability is another important difference. The GBDT model provided a clear feature-importance ranking, which made it easier to identify influential predictors such as previous campaign outcome, contact type, duration, and month-related variables. The MLP, while still effective, behaves more like a black box, making it harder to explain exactly which inputs drive its predictions. In the context of this dataset, interpretability is valuable because it helps explain which features were most associated with a customer subscribing to a term deposit.

Overall, GBDT was the stronger model for this task. Although the threshold-tuned MLP achieved the highest recall, GBDT still provided better accuracy, precision, F1 score, AUPRC, faster training time, and more interpretable outputs. For this structured tabular dataset, the tree-based approach appears to be the more suitable modeling strategy.

5 Discussion

The results suggest that GBDT was the more effective model overall for this term-deposit prediction task, although the tuned MLP achieved the highest recall. On the final test set, GBDT achieved an accuracy of 0.8969, precision of 0.5445, recall of 0.7251, F1 score of 0.6220, and AUPRC of 0.6070, while the MLP achieved an accuracy of 0.8888, precision of 0.5166, recall of 0.7667, F1 score of 0.6173, and AUPRC of 0.5823. These differences likely reflect how each model handled the tradeoff between finding more positive cases and avoiding false positives. After threshold

tuning, the MLP became more aggressive and captured more actual subscribers, but GBDT still maintained stronger overall balance according to F1 and AUPRC.

One interesting part of the results is that the MLP looked competitive earlier in the pipeline. Its baseline validation F1 was slightly higher than GBDT's (0.5621 vs. 0.5541), and its mean cross-validation F1 was also slightly higher (0.5630 +/- 0.0089 vs. 0.5543 +/- 0.0052). However, GBDT performed better on the final test set. One possible reason is that GBDT is often better suited for structured tabular data, where nonlinear feature interactions and threshold-based patterns can be captured effectively, allowing it to generalize well on unseen examples. Another reason is that the MLP search space was more limited. In the code, the MLP grid search only tested three architectures, two activation functions, three learning rates, and two iteration limits, while the GBDT search covered a broader set of hyperparameter combinations. The tuning outputs also suggest that the MLP was more sensitive to the exact selection criterion: for example, the deeper tanh network reached a validation F1 of 0.6016, but the cross-validated grid search selected a smaller ReLU network whose validation F1 was 0.5696. That mismatch suggests the neural network's performance was less stable across splits and tuning choices.

The bias-variance tradeoff also helps explain why the numbers look the way they do. For GBDT, the final tuned configuration had a training F1 of 0.7722 and a validation F1 of 0.5669, which shows a fairly large gap and indicates a higher risk of overfitting. Some of the strongest GBDT runs had training F1 values near 0.80 while validation F1 stayed around 0.57, which reinforces that point. At the same time, that extra flexibility likely helped GBDT capture useful minority-class structure that the MLP missed. In contrast, the final tuned MLP had a training F1 of 0.6142 and a validation F1 of 0.5696, which is a much smaller gap. This suggests that the MLP was more stable but also somewhat higher-bias: it generalized more conservatively before threshold tuning, but it may not have been expressive enough, or tuned broadly enough, to capture the strongest positive-class patterns as effectively as GBDT. The GBDT learning-rate experiment and max-depth validation curve (Figures 2 and 7) support this interpretation as well. With learning_rate = 0.01, the validation F1 dropped to only 0.1425, which suggests strong underfitting with the available number of trees. When the learning rate increased to 0.10 and 0.30, the validation F1 improved sharply to 0.5471 and 0.5671, showing that model capacity and fitting aggressiveness had a major effect on performance.

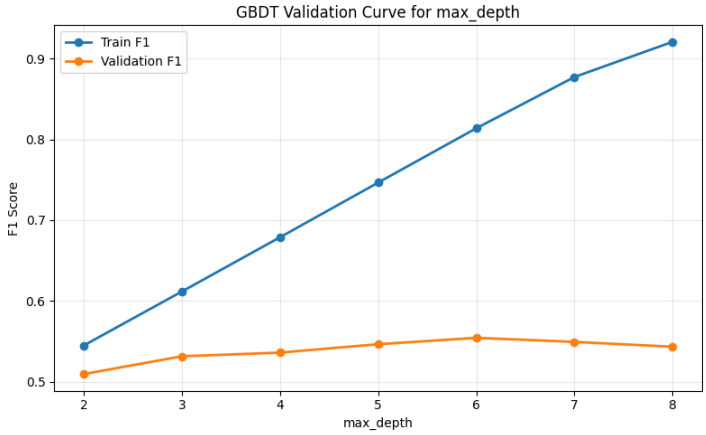

Figure 7: GBDT validation curve for max_depth.

Another important result comes from the GBDT feature-importance plot. The most important

feature was poutcome = success with an importance score of 103.71, followed by contact = MISSING (35.94), month = mar (29.14), duration (23.23), and housing = no (19.43). Other month indicators, including jun, oct, sep, apr, and feb, also appeared among the top features. This suggests that the model relied a lot on campaign history and timing, not just on customer background information. That is reasonable for this problem, since a customer who responded well in a previous campaign may be more likely to subscribe again, and month may reflect seasonal patterns or differences in campaign timing. Tree-based models often perform well with these kinds of clear split-based patterns. These results are useful, but they need to be interpreted in context. Duration stands out as an important feature, yet it would be less practical in a real-world prediction setting because it is only known after contact. In the same way, month and contact = MISSING likely reflect patterns in the campaign data rather than direct causes of subscription. Threshold tuning also influenced the final results. For both models, lowering the threshold from 0.50 to 0.30 increased recall and improved F1 score, although precision decreased. This is a reasonable tradeoff in an imbalanced classification setting where identifying the positive class is especially important.

6 AI Tool Disclosure

I used ChatGPT as a support tool during this assignment. It helped me review code structure, identify issues in the experimental setup, improve the clarity of my written explanations, and assist with generating the visualizations used in the analysis. In particular, it helped me notice methodological issues such as preprocessing leakage in cross-validation, the need to make the GBDT feature-importance plot more interpretable, and the need to tune the MLP decision threshold so the final comparison with GBDT was fairer. It also helped me think through and select a small set of engineered features to test in the final pipeline.

The main work of the project was still my own. I selected the Bank Marketing dataset, implemented the data cleaning and preprocessing pipeline, integrated the engineered features into the workflow, built and tuned the GBDT and MLP models, ran the experiments, and decided how to interpret and present the final results. Any AI-generated suggestions were reviewed and incorporated only after I checked them against my code, notebook outputs, and project goals.